



V1.0.0

MAIA — Documentação Técnica

Sistema single-tenant de saúde ocupacional e
segurança do trabalho

19 de maio de 2026

Cliente: ENGEKO ENGENHARIA E CONSTRUÇÃO LTDA.

Entregue por HEIZEN TECNOLOGIA LTDA. · CNPJ 47.624.793/0001-83 ·

fapptory.me

MAIA — Documentação Técnica

Sistema single-tenant de saúde ocupacional e segurança do trabalho da **ENGEKO**, distribuído em dois repositórios (`maia-db` para o backend Supabase, `maia-app` para o frontend Next.js). Esta documentação cobre o sistema como um todo na versão **v1.0.0** (entregue em 2026-05-19).

Sumário

- [1. Visão geral](#)
- [2. Arquitetura](#)
- [3. Stack técnica](#)
- [4. Estrutura dos repositórios](#)
- [5. Modelo de dados](#)
- [6. Autenticação e autorização](#)
- [7. Fluxos principais](#)
- [8. Integrações externas](#)
- [9. Rotas e endpoints](#)
- [10. Sistema de e-mails](#)
- [11. Auditoria \(eventos\)](#)
- [12. Design system](#)
- [13. Testes](#)
- [14. Variáveis de ambiente](#)
- [15. Deploy](#)
- [16. Convenções](#)
- [17. Backlog e oportunidades de melhoria](#)

1. Visão geral

A **MAIA** é um sistema de uso interno personalizado para a ENGEKO, para gestão de **afastamentos de colaboradores** e **ocorrências de segurança do trabalho**. É um produto single-tenant — uma única empresa-usuário.

Objetivos do produto

- **Coletar** afastamentos via formulário público (sem necessidade de login do colaborador).
- **Aprovar** afastamentos médicos por uma equipe interna de Saúde Ocupacional (`oh`).
- **Transferir** afastamentos aprovados para o Fluig (TOTVS), o ERP de RH/Folha da ENGEKO.
- **Notificar** colaboradores, gestores e a folha de pagamento por e-mail.

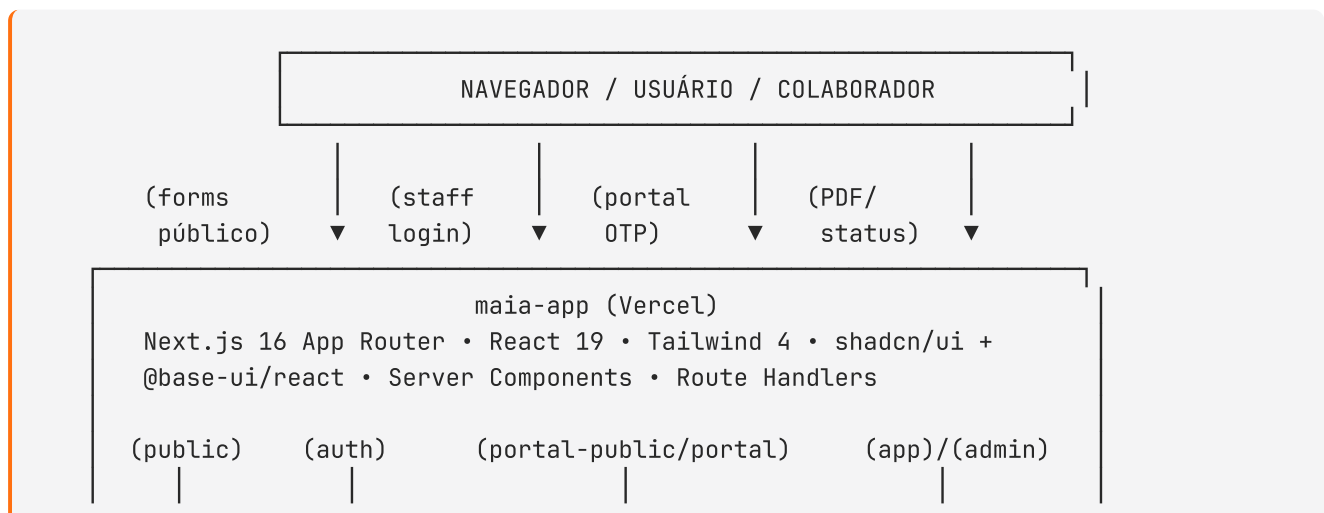
- **Permitir resubmissão** quando um afastamento for rejeitado (via token público estável).
- **Registrar e investigar ocorrências** de segurança — com formulário Ishikawa (6Ms) completo, ações corretivas, fotos e relatório PDF.
- **Portal do colaborador** — login por OTP+CPF (sem senha, sem Supabase Auth) para que o autor consulte o histórico das próprias submissões e do colaborador.
- **Comentar** afastamentos internamente entre membros da Saúde Ocupacional, com anexos.
- **Acompanhar a saúde da operação** via painel com métricas de falhas e latência.
- **Exportar** afastamentos e ocorrências em CSV para análises externas.

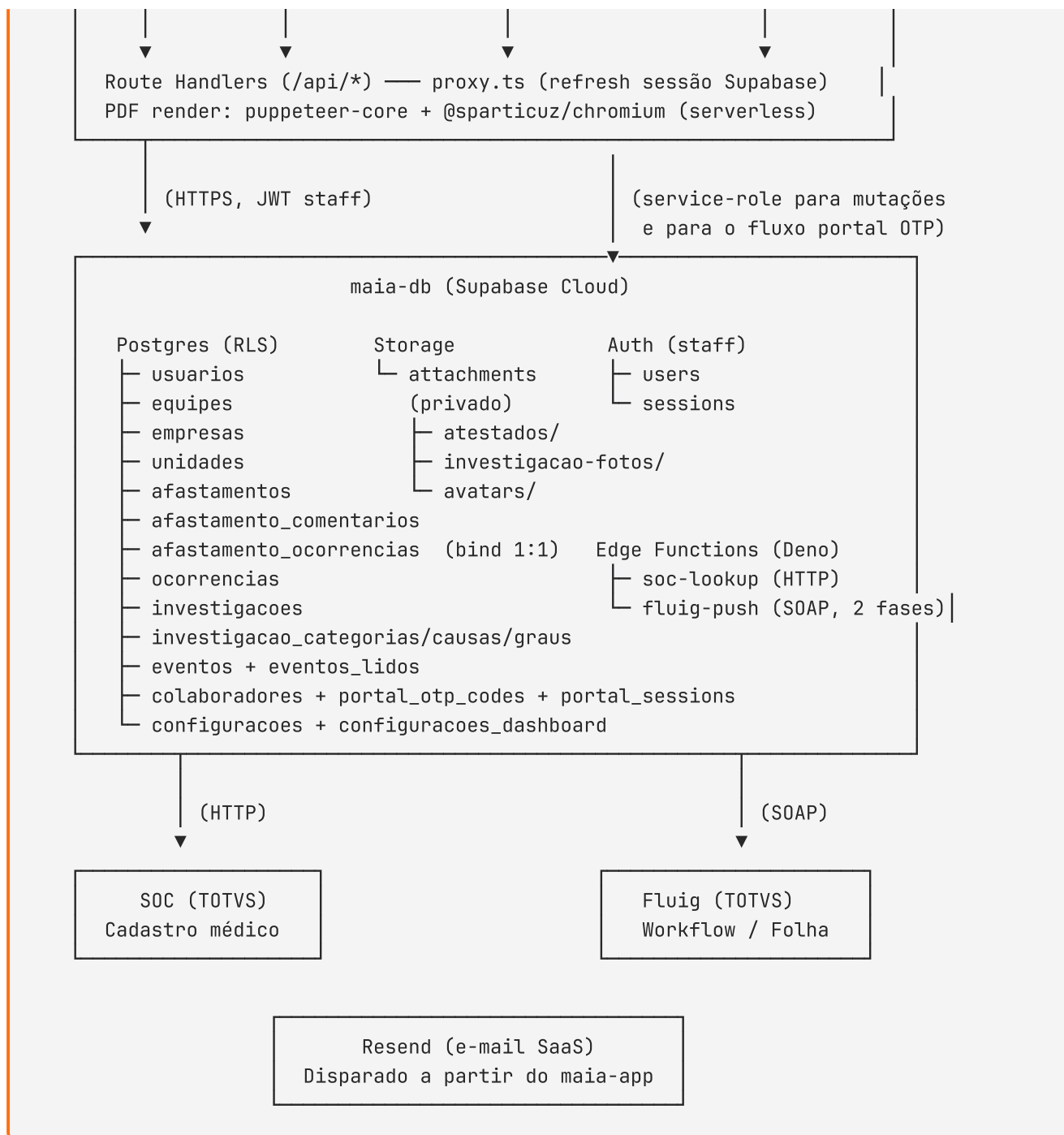
Decisões de design importantes

- **Idioma do domínio:** tudo em **português do Brasil** — nomes de tabelas, colunas, rótulos, textos de UI, documentação.
- **State machines explícitas:** ciclo de vida de afastamento (`pendente` → `finalizado/rejeitado/cancelado`), de ocorrência (`aberta` → `em_investigacao` → `concluida`) e de investigação (`em_andamento` → `em_aprovacao` → `aprovada/rejeitada/cancelada`) validados em código.
- **Trilha de auditoria unificada:** uma única tabela `eventos` registra qualquer mudança relevante em qualquer entidade. Uma tabela acessória `eventos_lidos` rastreia o read-receipt por usuário.
- **Tokens públicos estáveis:** links estilo Calendly/DocuSign permitem que o autor reenvie um registro rejeitado, preencha uma investigação, ou gere um relatório PDF de ocorrência sem fazer login.
- **Portal do colaborador desacoplado do Supabase Auth:** o colaborador autentica por CPF + código OTP de 6 dígitos enviado por e-mail; sessões ficam em tabela dedicada (`portal_sessions` , cookie `portal_session`). Não há linha em `auth.users` para colaboradores — apenas para staff interno.

2. Arquitetura

Diagrama em alto nível





Responsabilidades por camada

Camada	Onde fica	Responsabilidades
UI staff	maia-app/app/(app) , app/(admin) , components/	Painéis, listas, aprovações, perfil, saúde da operação
UI pública	maia-app/app/(public)	Formulários, status, edição por token, relatório PDF público
UI portal colaborador	maia-app/app/(portal-public) , app/(portal)	Login OTP, painel pessoal, histórico de afastamentos

Camada	Onde fica	Responsabilidades
API interna	<code>maia-app/app/api/</code>	Route Handlers Next.js (autenticados, públicos e do portal)
Lógica de domínio	<code>maia-app/lib/</code>	State machines, permissões, validação Zod, helpers de eventos, sessões do portal
Camada de dados	<code>maia-db/supabase/migrations/</code>	DDL, índices, triggers, políticas RLS, seeds
Integrações Deno	<code>maia-db/supabase/functions/</code>	<code>soc-lookup</code> (cadastro SOC), <code>fluig-push</code> (workflow TOTVS via SOAP)
Auth (staff)	Supabase Auth	Usuários internos, sessões JWT, SMTP customizado para invites
Auth (portal)	<code>lib/portal-*</code> + tabelas <code>portal_*</code>	OTP por e-mail, sessão por cookie próprio (<code>portal_session</code> , 7 dias)
Armazenamento	Supabase Storage (bucket <code>attachments</code>)	Atestados, fotos de investigação, avatares (uploads via service-role)
E-mail	Resend + templates HTML do <code>maia-app</code>	E-mails transacionais (recibo, aprovação, rejeição, OTP, notificações à folha/safety)
PDF	<code>puppeteer-core</code> + <code>@sparticuz/chromium</code>	Renderização server-side de relatórios de ocorrência (no runtime do Vercel)

3. Stack técnica

maia-app (frontend)

- **Next.js 16** (App Router, Server Components, Route Handlers) — note o uso de `proxy.ts` em lugar de `middleware.ts` (renomeação Next.js 16).
- **React 19**
- **TypeScript** estrito
- **Tailwind CSS 4** (com `@theme inline` e tokens em `app/tokens.css`)

- **shadcn/ui** (primitivas Radix instaladas sob demanda) + **@base-ui/react** (para componentes mais sofisticados como `Select`, `Sheet`, `Dialog`, `Combobox` — adotado durante o desenvolvimento do portal e do form Ishikawa)
- **@supabase/ssr** + **@supabase/supabase-js** (clientes server, browser e admin)
- **Resend v6.12** (SDK Node para e-mail)
- **zod v4** + **react-hook-form** + **@hookform/resolvers** (validação)
- **sonner** (toasts), **lucide-react** (ícones), **clsx** + **tailwind-merge** + **class-variance-authority** (helpers shadcn)
- **date-fns** (datas), **tw-animate-css** (animações)
- **puppeteer-core** + **@sparticuz/chromium** (geração de PDF server-side)
- **react-resizable-panels** (inbox redimensionável de aprovações)
- **Vitest 4** (unitário) + **Playwright** (E2E)
- **tsx** (executor TS para scripts CLI — ex: `scripts/migrate-afastamentos.ts`)

maia-db (backend)

- **Supabase Postgres** (Postgres 15+) — extensão `pgcrypto` para `gen_random_uuid()`
- **Supabase CLI** + **Make** para deploy/reset/secrets
- **Deno** + edge runtime do Supabase (`jsr:@supabase/functions-js/edge-runtime.d.ts`)
- Sem framework adicional: edge functions são `Deno.serve(...)` puro

Sem

- Sem React Email (templates são strings HTML com CSS inline)
- Sem Radix UI instalado diretamente (apenas via shadcn add) — **@base-ui/react** é a alternativa adotada para componentes que não vêm do shadcn
- Sem pnpm/yarn/bun (apenas npm)
- Sem dotenv (Next.js carrega `.env.local` nativamente)
- Sem CI/CD configurado neste momento (deploy manual via Vercel + Supabase CLI)

4. Estrutura dos repositórios

Layout do filesystem

```
/Users/heizen/DEV/  
├── maia-db/           ← migrações + edge functions  
└── maia-app/         ← frontend Next.js
```

maia-db

```
maia-db/  
├── Makefile           # db-reset, db-push, functions-deploy,
```

```

secrets
├── README.md
├── .env.example
├── supabase/
│   ├── config.toml
│   └── seed.sql # fixtures de desenvolvimento (usuários,
colaborador, # afastamentos, empresas/unidades) – NÃO
é uma migration
├── migrations/
│   ├── 001_extensions.sql # pgcrypto + set_atualizado_em()
│   └── 002_usuarios.sql # + handle_new_auth_user() (trigger auth
→ public)
├── 003_equipes.sql # equipes (oh, safety) + equipe_usuarios
├── 004_configuracoes.sql # singleton (id = 1) – email_folha
├── 005_empresas.sql
├── 006_unidades.sql
├── 007_afastamento_tipos.sql # 12 tipos seedados
├── 008_afastamentos.sql # core + serial_id + token_edicao
├── 009_ocorrencias.sql # + serial_id + token_publico
├── 010_investigacoes.sql # state machine 5 estados + token_publico
├── 011_eventos.sql # auditoria unificada
├── 012_eventos_lidos.sql # read receipts (usuario_id, evento_id)
├── 013_storage.sql # bucket attachments (privado)
├── 014_rls.sql # is_admin/is_in_equipe + políticas
SELECT
├── 015_investigacao_taxonomy.sql # 6 categorias + 3 graus + 83 causas
seedadas (Ishikawa)
├── 016_configuracoes_dashboard.sql # singleton para tuning
(aprovacao_lenta_horas)
├── 017_colaboradores_portal.sql # colaboradores + portal_otp_codes +
portal_sessions
├── 018_afastamento_ocorrencias.sql # bind 1:1 (acidente → afastamento)
├── 019_afastamento_comentarios.sql # comentários com anexos jsonb
├── 020_usuarios_perfil.sql # usuarios.primeiro_acesso +
usuarios.avatar_url
├── tests/
│   └── rls.sql # smoke psql para RLS
├── functions/
│   ├── _shared/
│   │   ├── env.ts # requireEnv()
│   │   ├── types.ts # mapTipoToFluigCode + FluigPushPayload
│   │   ├── deno.json
│   │   └── fluig-mapping.test.ts # 4 testes Deno
│   ├── soc-lookup/
│   │   ├── index.ts # POST → consulta CPF no SOC (ISO-8859-1)
│   │   ├── deno.json
│   │   └── test.ts
│   └── fluig-push/
│       ├── index.ts # SOAP 2-fases: createSimpleDocument +
startProcess
└── deno.json

```

maia-app

```

maia-app/
├── package.json                # 22 prod deps + 11 dev deps
├── tsconfig.json
├── next.config.ts
├── postcss.config.mjs
├── eslint.config.mjs
├── components.json            # shadcn
├── vitest.config.ts           # alias @/ + exclude tests/e2e
├── playwright.config.ts
├── proxy.ts                   # Next.js 16: refresh de sessão + gate
└── /app/*
    ├── .env.example
    ├── docs/
    │   ├── DOCUMENTACAO.md    # este arquivo
    │   └── ENTREGA-v1.0.0.md  # termo de entrega final (não editar)
    ├── scripts/
    │   ├── migrate-afastamentos.ts # CLI tsx: migrar legado para nova base
    │   └── output/            # HTMLs gerados (user-invite, password-
└── reset)
    ├── public/                # logos ENGEK0 e Fapptory, favicons
    └── app/
        ├── layout.tsx         # lang="pt-BR", Toaster sonner
        ├── globals.css        # imports tokens.css + Tailwind
        ├── tokens.css         # CSS variables (cor/espaco/tipografia)
        ├── page.tsx           # raiz (linktree público)
        ├── (auth)/            # login staff
        │   ├── login/page.tsx
        │   ├── forgot-password/page.tsx
        │   └── update-password/page.tsx
        ├── auth/
        │   ├── callback/route.ts # callbacks Supabase Auth
        │   ├── PKCE exchange
        │   ├── verify OTP token_hash
        │   └── # rotas públicas (sem login)
        ├── (public)/
        │   ├── forms/
        │   │   ├── afastamentos/page.tsx
        │   │   └── ocorrencias/page.tsx
        │   ├── afastamentos/
        │   │   ├── editar/[token]/page.tsx # reenvio
        │   │   └── status/[token]/page.tsx # consulta por protocolo
        │   ├── ocorrencias/
        │   │   ├── status/[token]/page.tsx
        │   │   └── relatorio/[token]/page.tsx # relatório PDF público
        │   ├── investigacoes/
        │   │   └── editar/[token]/page.tsx # preenchimento de investigação por token
        ├── (portal-public)/
        │   └── portal/
        │       ├── login/page.tsx        # form CPF+email → OTP
        │       └── cadastro/page.tsx
        └── (portal)/
            # portal – autenticado por cookie
portal_session
├── |
└── | └── portal/

```


	layout.tsx	# gate requirePortalSession()
	painel/page.tsx	# status do colaborador (afastado /
livre)		
	afastamentos/[id]/page.tsx	# detalhe acessível ao próprio
colaborador		
	(app)/	# gate: usuário autenticado (staff)
	layout.tsx	# TopNav + Notification bell
	app/	
	painel/	
	page.tsx	# contadores + atividade recente
	saude/page.tsx	# painel de saúde da operação
	perfil/page.tsx	# nome/senha/avatar
	afastamentos/	
	page.tsx	# lista com filter rail
	ativos/page.tsx	# filtro pronto (finalizados ativos)
	[id]/page.tsx	# detalhe + timeline + comentários
	aprovacoes/page.tsx	# inbox OH (resizable)
	ocorrencias/	
	page.tsx	
	[id]/page.tsx	
	[id]/investigacao/page.tsx	# form Ishikawa completo
	investigacoes/page.tsx	# lista para equipe safety
	admin/	# /app/admin/*
	page.tsx	# hub
	usuarios/page.tsx	
	equipes/page.tsx	
	configuracoes/page.tsx	
	empresas/page.tsx	
	unidades/page.tsx	
	afastamento-tipos/page.tsx	
	colaboradores/page.tsx	# consulta cadastro/histórico
	investigacao/	
	categorias/page.tsx	
	causas/page.tsx	
	graus/page.tsx	
	api/	# ver §9 para lista completa
	components/	
	ui/	# shadcn primitives + @base-ui/react
adapters		
	layout/	# app-top-nav, app-notification-bell,
etc.		
	home/	# linktree público (landing)
	auth/	# auth-card
	gates/equipe-only.tsx	# requireEquipe('oh' 'safety')
	data/	# data-table, empty-state, filter-rail,
status-pill		
	detail/	# field-grid, timeline-events, stepper,
attachment-chip, approval-bar		
	forms/	# afastamento-form, ocorrencia-form, cpf-
lookup, file-upload, public-form-shell, form-errors		
	admin/crud-table.tsx	# tabela genérica
	afastamentos/	# detail, history, aprovacoes-panel,
comentarios-card, comentario-dialog, edit-dialog, export-history		
	ocorrencias/	# detail-card

```

|   |— investigacoes/                                # form (Ishikawa), report (PDF source),
branch editor, action-item editor, foto-uploader, participante-list, data-view,
detail-section, decision-action-bar
|   |— portal/                                        # home-button, logout-button
|   |— painel/                                        # hero, kpi-card, activity-feed, quick-
action, colaborador-summary-card
|   |— saude/                                          # saude-client, saude-banner, metric-
card, fluig-error-sheet
|   |— notifications/                                # notification-dialog, notification-item
|   |— relatorios/                                   # export-dialog
|   |— brand/                                         # logo, fapptory-attribution
|— lib/
|   |— supabase/
|   |   |— server.ts                                # cookies-scoped (server components,
route handlers)
|   |   |— client.ts                                # browser
|   |   |— admin.ts                                 # service-role (server-only, bypassa RLS)
|   |   |— database.types.ts                        # AUTO-GERADO via `supabase gen types
typescript --linked`
|   |— mail/send.ts                                  # Resend + registro de 14 templates
|   |— data/                                          # cids.json, ufs.json,
ocorrencia_tipos.json
|   |— dashboard/queries.ts                          # métricas para /api/saude
|   |— relatorio/                                    # csv builder + afastamentos-csv +
ocorrencias-csv
|   |— validation/                                   # AfastamentoInputSchema +
OcorrenciaInputSchema
|   |— eventos.ts                                    # writeEvento() + 13 EventoTypes
|   |— eventos-format.ts                             # rótulos PT-BR para eventos
|   |— soc.ts                                         # invoca soc-lookup edge fn
|   |— fluig.ts                                       # invoca fluig-push edge fn (nunca lança
- retorna {ok,error?})
|   |— permissions.ts                                # isAdmin, isInEquipe
|   |— admin-auth.ts                                 # requireAdminUser()
|   |— current-user.ts                               # carrega Me com equipes
|   |— afastamento-state.ts                         # canTransition + isEditAllowed
|   |— afastamento-date.ts                         # cálculo de data_fim por tipo
|   |— afastamento-tipo-rules.ts                   # regras de campo por tipo
|   |— ocorrencia-state.ts                          # states + labels de tipo
|   |— investigacao-state.ts                        # 5 situacoes + 4 planos de ação
|   |— investigacao-dados.ts                        # schema jsonb da investigação +
sanitizer
|   |— investigacao-fk-check.ts                     # valida FKs dentro do dados jsonb
|   |— investigacao-step-gates.ts                   # gates por step da investigação
|   |— portal-auth.ts                               # requirePortalSession() via cookie
|   |— portal-session.ts                            # CRUD de portal_sessions (7 dias)
|   |— portal-status.ts                             # findActiveAfastamento() do colaborador
|   |— safety-notify.ts                             # notifica equipe safety quando
ocorrência é criada
|   |— auth-errors.ts                               # mensagens padronizadas
|   |— auth-schemas.ts                             # zod schemas (login, OTP, magic link,
reset)
|   |— colaborador-summary.ts                       # display CPF/nome
|   |— filter-rail.ts                               # filter rail query builder

```

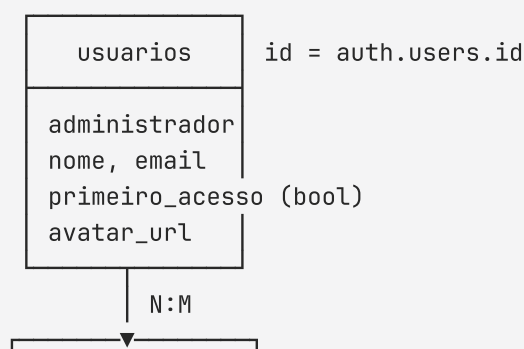
```

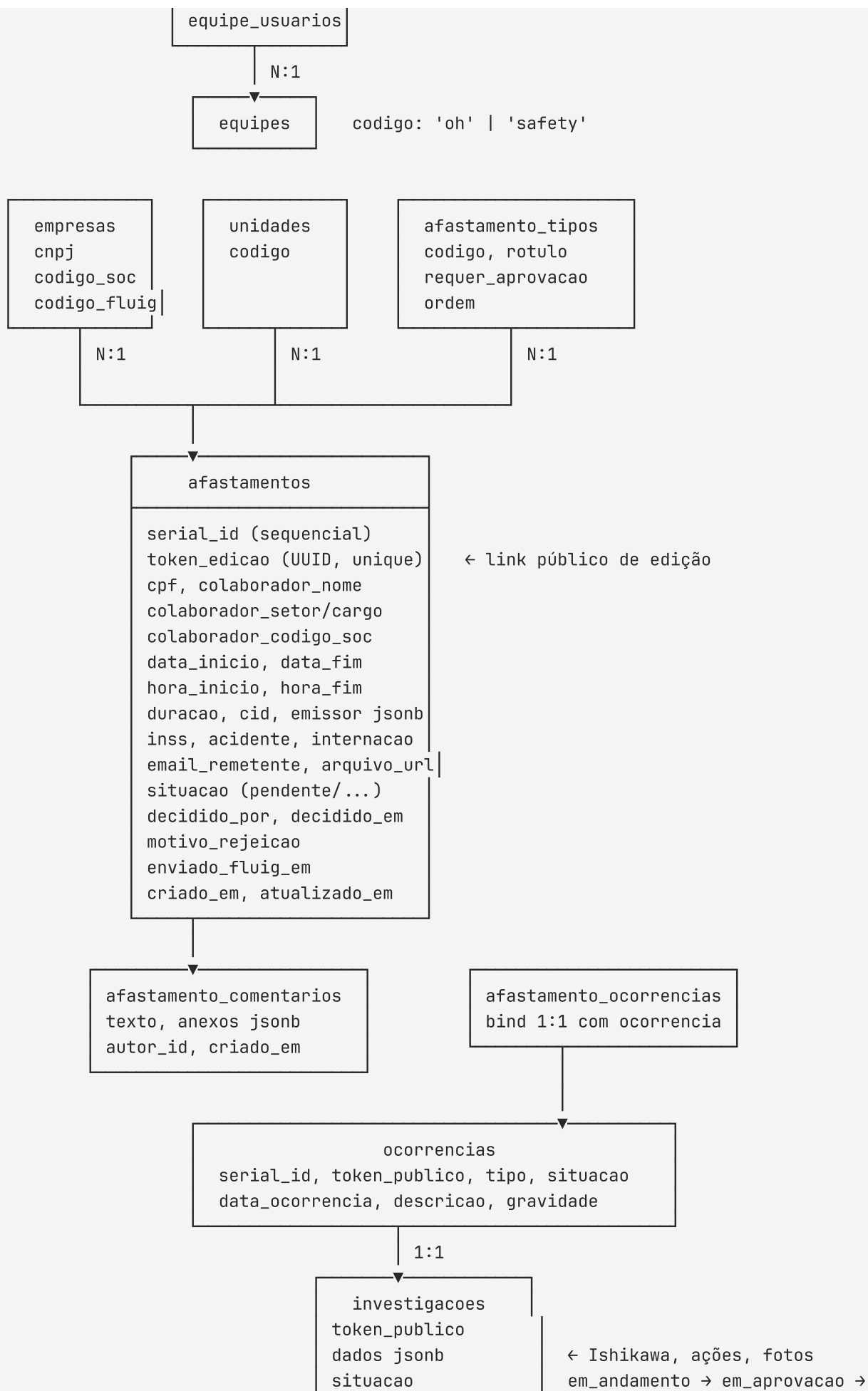
├── fmt-date.ts           # formatadores PT-BR
├── greeting.ts           # saudação contextual (bom dia/tarde)
├── nav.ts                 # navegação interna
├── public-links.ts        # URL builders para forms públicos
├── public-nav.ts          # navegação pública
├── status-pill.ts         # variantes de pill por situacao
├── painel-hero-content.ts # hero card por papel
├── version.ts             # leitura do package.json
(NEXT_PUBLIC_APP_VERSION)
├── fapptory.ts           # watermark/branding utilitário
├── utils.ts               # cn(...)
├── emails/               # 14 templates registrados + utilitários
│   ├── _escape.ts
│   ├── _layout.ts
│   ├── _record-table.ts
│   ├── tokens.ts         # cores email-safe
│   ├── afastamento-receipt.ts
│   ├── afastamento-rejected.ts
│   ├── afastamento-approved.ts
│   ├── folha-auto-accept.ts
│   ├── folha-approved-medical.ts
│   ├── ocorrencia-receipt.ts
│   ├── ocorrencia-nova-para-safety.ts # notifica equipe safety
│   ├── investigacao-em-aprovacao.ts
│   ├── investigacao-aprovada.ts
│   ├── investigacao-rejeitada.ts
│   ├── portal-otp.ts     # OTP de 6 dígitos
│   ├── magic-link.ts     # alternativa de acesso staff
│   ├── user-invite.ts    # colado no Supabase Dashboard
│   ├── password-reset.ts # colado no Supabase Dashboard
│   └── relatorio-pronto.ts # arquivado (não registrado em TEMPLATES;
reservado para evolução)
├── tests/
│   ├── unit/             # 35 arquivos Vitest – ver §13
│   └── e2e/              # 5 arquivos Playwright

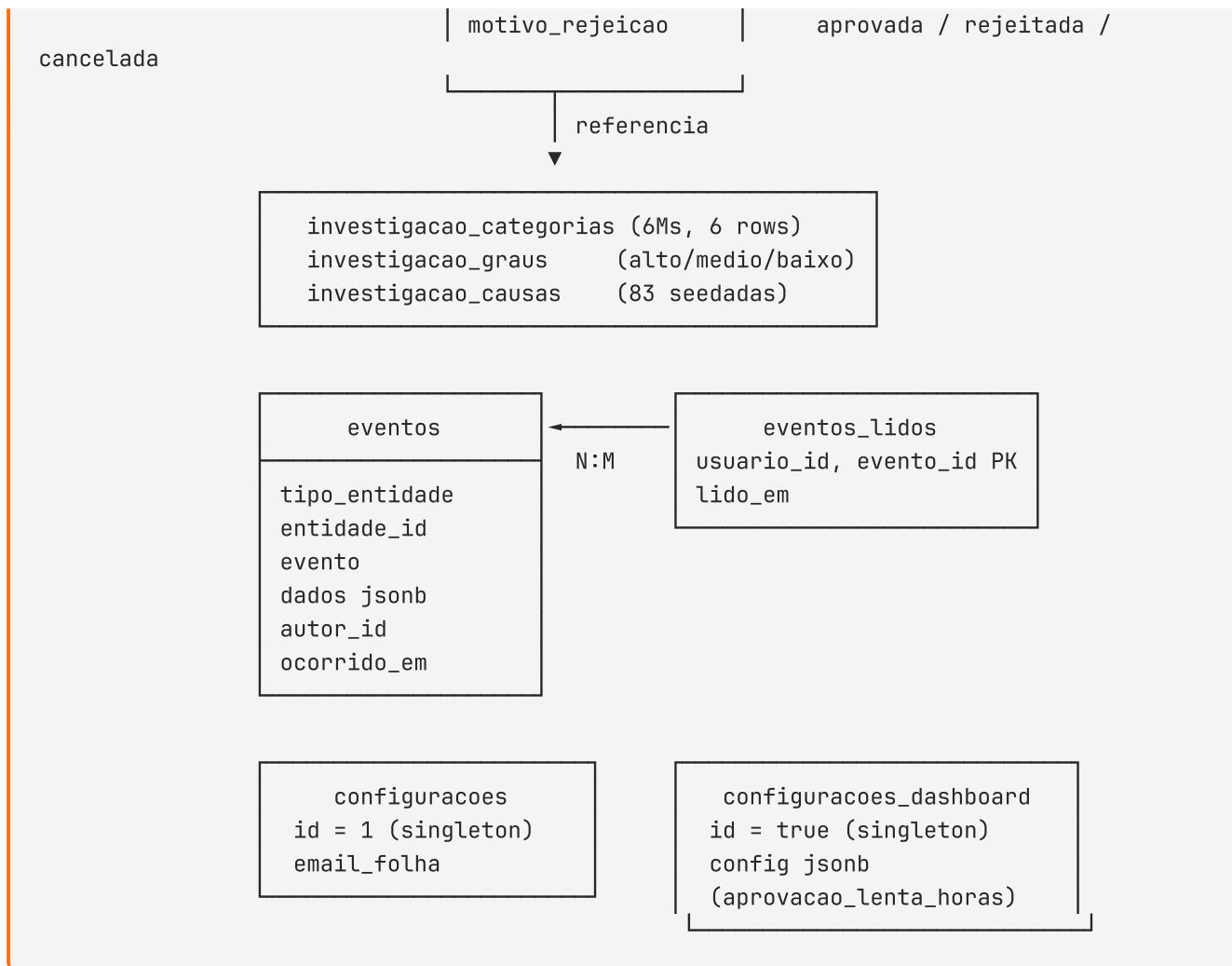
```

5. Modelo de dados

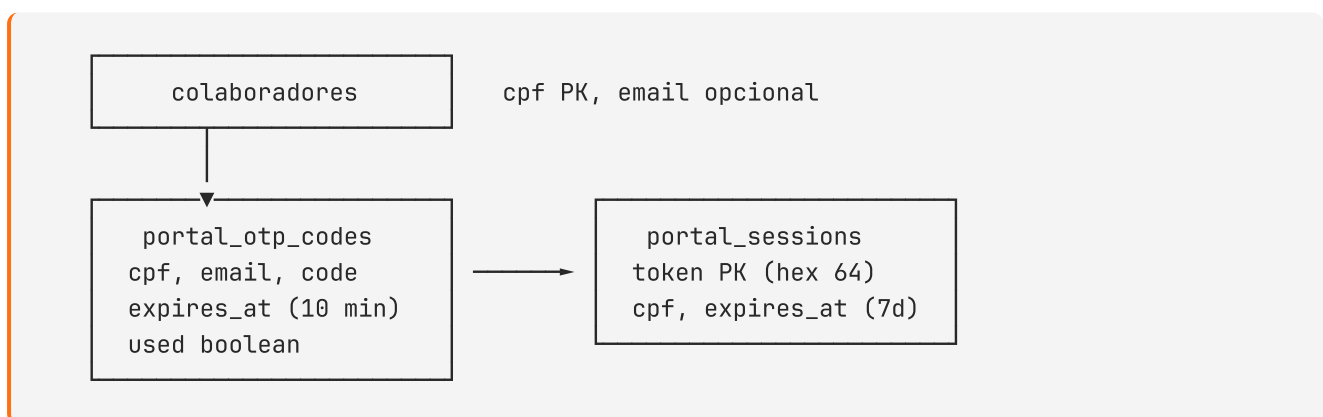
Tabelas principais (staff e domínio)







Subsistema portal do colaborador (desacoplado de auth.users)



Convenções de schema

- **Identificadores em PT-BR:** usuarios, equipes, afastamentos, ocorrencias, situacao, criado_em, atualizado_em, etc.
- **PKs:** gen_random_uuid() por padrão; exceções: usuarios.id espelha auth.users.id; configuracoes.id = 1; configuracoes_dashboard.id = true; colaboradores.cpf é PK textual; portal_sessions.token é PK textual.

- **Identificadores humanos:** `afastamentos.serial_id` e `ocorrencias.serial_id` são sequenciais (visíveis nos assuntos de e-mail como `#123`).
- **Timestamps:** `criado_em timestampz not null default now() + atualizado_em timestampz` mantida via trigger `set_atualizado_em()` (definido em `001_extensions.sql`).
- **Soft delete:** campo `ativo boolean` nas tabelas administrativas. Sem `DELETE` real.
- **State machines em CHECK:** a coluna `situacao` tem `check (situacao in (...))` no DDL; a aplicação valida transições via `lib/afastamento-state.ts` , `lib/ocorrencia-state.ts` e `lib/investigacao-state.ts` .
- **Tokens públicos:** `afastamentos.token_edicao` , `ocorrencias.token_publico` , `investigacoes.token_publico` são UUIDs únicos usados para acesso sem login.

Bucket de storage

- `attachments` (privado): recebe anexos de afastamentos, fotos de investigação, avatares de perfil. Upload feito server-side via `service-role`; download via URL assinada (`/api/private/anexos/preview` e variantes públicas por token). Sem políticas RLS em `storage.objects` — controle todo na aplicação.
- `support` (público): destinado a upload e compartilhamento de imagens e arquivos de suporte ao aplicativo.

Tabelas auxiliares e padrões

- `eventos_lidos` : read-receipt por usuário, usado pelo sino de notificações (`/api/notificacoes`). PK composta (`usuario_id` , `evento_id`) ; ON DELETE CASCADE em ambos os lados.
- `investigacao_*` : taxonomia editável; `categorias` tem 6 Ms seedados, `graus` 3 níveis, `causas` 83 entradas curadas. O form Ishikawa lê dessas tabelas; usuários podem digitar causas livremente.
- `afastamento_comentarios` : `texto` + `anexos jsonb` (lista de `{ nome, url, content_type, tamanho }`). Anexos são uploads para o bucket `attachments` .
- `afastamento_ocorrencias` : PK = `afastamento_id` e UNIQUE em `ocorrencia_id` ⇒ relação 1:1 (uma ocorrência gera no máximo um afastamento e vice-versa).

6. Autenticação e autorização

A MAIA tem **duas árvores de autenticação independentes**:

1. **Staff** (Saúde Ocupacional, Segurança, Admin) — Supabase Auth (email + senha).
2. **Colaborador** (portal) — OTP por e-mail + CPF, sem coupling com `auth.users` .

6.1 Staff (Supabase Auth)

1. **Supabase Auth** controla identidade. Usuários são criados via `inviteUserByEmail` a partir do admin; não há autoinscrição.
2. `proxy.ts` (Next.js 16 — antes `middleware.ts`) refresca a sessão a cada request. Se o caminho começa com `/app/` e não há `user` , redireciona para `/login` . As rotas públicas (`_next/static` ,

`_next/image`, `favicon.ico`, `forms/*`, `api/public/*`) são excluídas via matcher.

3. Layouts gateadores:

- `app/(app)/layout.tsx` redireciona para `/login` se não houver usuário.
- `app/(admin)/layout.tsx` adicionalmente redireciona para `/painel` se `administrador` `=== true`.

4. Helpers de aplicação:

- `lib/admin-auth.ts` → `requireAdminUser()` para route handlers que mutam dados administrativos.
- `components/gates/equipe-only.tsx` → `requireEquipe('oh' | 'safety')` para páginas.
- `lib/permissions.ts` → `isAdmin(me)`, `isInEquipe(me, codigo)` para checagens síncronas.

5. **Primeiro login:** usuários convidados têm `usuarios.primeiro_acesso = true`. Após definirem senha em `/update-password`, o flag é zerado pelo handler.

6. **RLS no Postgres** (`014_rls.sql` + políticas em migrations subsequentes) atua como última linha de defesa para leitura:

- `is_admin(uid)` e `is_in_equipe(uid, codigo)` são funções `security definer`.
- Políticas `SELECT` em todas as tabelas; **mutações via service-role** após permission check no handler.
- Tabelas do portal (`colaboradores`, `portal_*`), de config dashboard e de comentários têm RLS habilitado mas **sem políticas** — `service-role-only` por padrão (defesa em profundidade).

6.2 Portal do colaborador (OTP + CPF)

Fluxo desacoplado do Supabase Auth. Implementado em `app/(portal-public)`, `app/(portal)` e `lib/portal-*`.

1. **Entrada** — `/portal/login` : usuário fornece CPF (11 dígitos) e e-mail.

2. **Validação do CPF** — `POST /api/portal/login-init` :

- Existe linha em `colaboradores` para o CPF? Se sim e tem e-mail cadastrado, ele precisa bater.
- Caso não exista em `colaboradores`, exige histórico em `afastamentos` (autor de pelo menos uma submissão).

3. **Geração de OTP** — gera código de 6 dígitos via `randomInt(100000, 999999)`. Reusa código ativo se existir (refresca expiração para 10 min); senão invalida códigos antigos e cria novo.

4. **E-mail** — `sendMail({ template: "portal-otp", ... })` despacha o código.

5. **Verificação** — `POST /api/portal/login-verify` : valida o código contra `portal_otp_codes` (constant-time compare), marca como `used = true`, cria linha em `portal_sessions` com token hex de 32 bytes e TTL de 7 dias.

6. **Cookie** — handler seta `portal_session=<token>` como `HttpOnly + Secure + SameSite=Lax`. As rotas `/portal/*` (autenticadas) usam `requirePortalSession()` que valida o cookie e devolve `{ cpf }`.

7. **Logout** — `POST /api/portal/logout` deleta a linha em `portal_sessions` e limpa o cookie.

Importante: o portal opera totalmente em service-role no servidor. Não há JWT Supabase para o colaborador, e nenhuma RLS é aplicada — toda a checagem é no handler (`requirePortalSession()` → service-role).

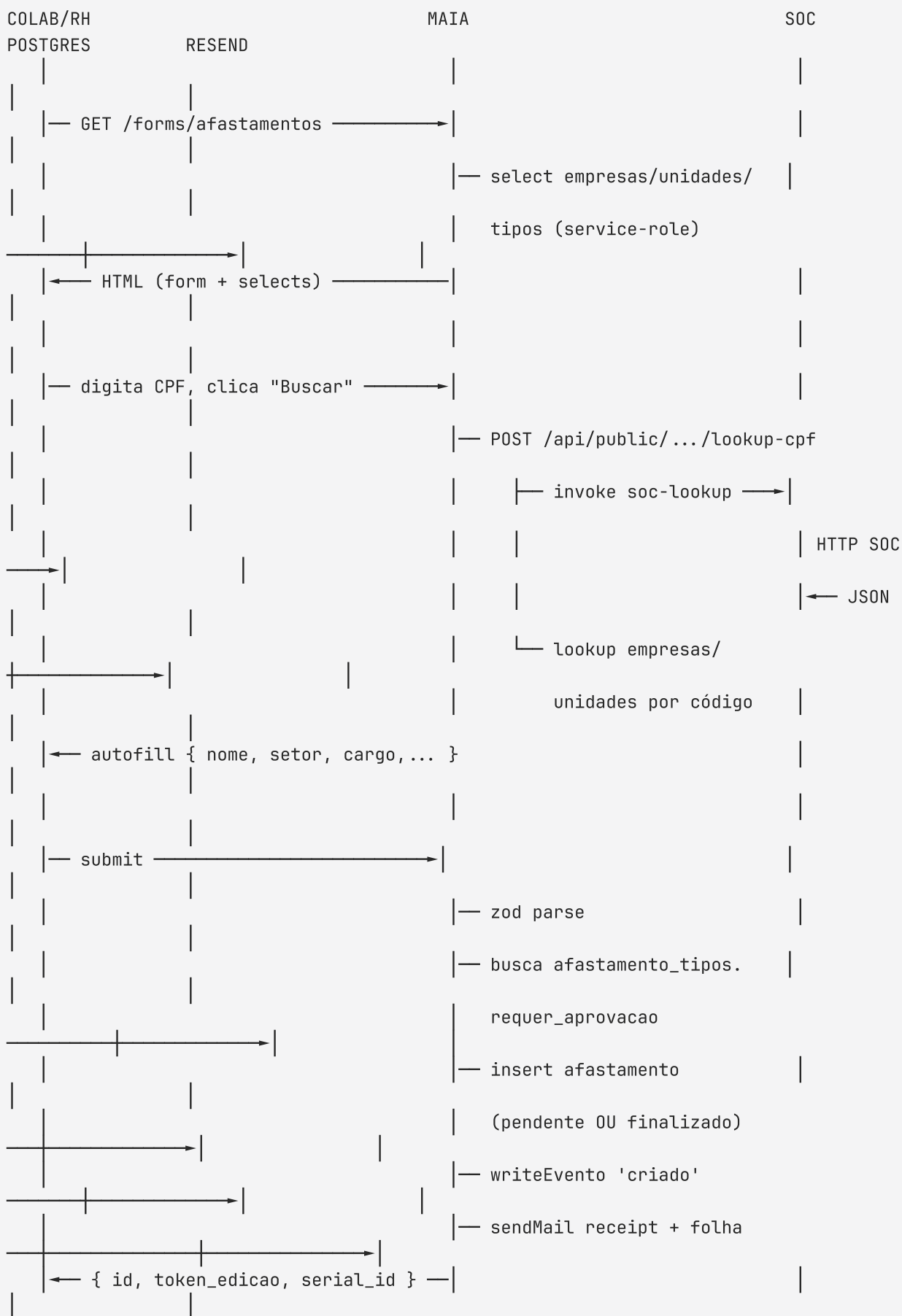
Modelo de papéis

Papel	Como é representado	O que pode fazer
Anônimo público	Sem sessão	Submeter afastamento/ocorrência via <code>forms/*</code> ; editar afastamento rejeitado, preencher investigação ou baixar relatório de ocorrência via token público; consultar status por protocolo.
Colaborador (portal)	Cookie <code>portal_session</code> válido	Ver o próprio histórico de afastamentos em <code>/portal/painel</code> e detalhe em <code>/portal/afastamentos/[id]</code> .
Staff comum	Linha em <code>usuarios</code> com <code>administrador = false</code>	Acessar <code>/app/painel</code> ; ver detalhes de afastamentos/ocorrências (sujeito a RLS por equipe).
OH (oh)	<code>equipe_usuarios.codigo = 'oh'</code>	Tudo de "Comum" + inbox de aprovações + aprovar/rejeitar afastamentos + comentar.
Safety (safety)	<code>equipe_usuarios.codigo = 'safety'</code>	Tudo de "Comum" + conduzir investigações (Ishikawa, plano de ação, aprovação).
Admin	<code>usuarios.administrador = true</code>	Tudo do sistema + CRUDs administrativos + cancelamento + retry Fluig manual + taxonomia Ishikawa.

Nota: as equipes são fixas — `oh` e `safety` são seedadas em `003_equipes.sql` .

7. Fluxos principais

7.1 Submissão de afastamento (público)



Tipos com `requer_aprovacao = true` (Doença, Acidente, INSS etc.) entram como **pendente** e vão para a inbox da SO.

Tipos com `requer_aprovacao = false` (Casamento, Óbito, Maternidade etc.) entram direto como **finalizado** e disparam um e-mail à folha.

7.2 Aprovação OH (com retry manual em caso de falha)



```

|
| se Fluig falhou, painel de saúde mostra registro em "Fluig falhados"
|
| — (admin only) abre FluigErrorSheet, clica "Retentar agora" —————>
|
|                                     | POST /api/afastamentos/[id]/fluig/retry
|                                     |
|                                     | — requireAdminUser
|                                     |
|                                     | — select afastamento (deve ser
|                                     |   finalizado + requer_aprovacao)
|                                     |
|                                     | — pushToFluig (idempotente)
|                                     |
|                                     | — writeEvento 'fluig_enviado'
|                                     |
|                                     |   ou 'fluig_erro' com retry:true
|
| — { ok: true } ou 502 —————>
|

```

Invariantes importantes:

- `fluig.ts` → `pushToFluig` **nunca lança** — sempre retorna `{ ok: boolean, error?: { message, body?, status? } }`. Isso evita derrubar a aprovação em caso de Fluig indisponível.
- Permissão é verificada **antes** de qualquer uso do `service-role`.
- Aprovação **prossegue mesmo se Fluig falhar** — registra `fluig_erro` em `eventos` e admin pode retentar depois pelo painel de saúde.
- Falhas de e-mail são engolidas (try/catch) e registradas como eventos `email_enviado` com `dados.error`.

7.3 Rejeição com link de edição

```

OH          MAIA          COLABORADOR
|           |             |
| — Rejeitar —————> | POST /api/afastamentos/[id]/rejeitar { motivo }
|                   | — validações + canTransition (→ rejeitado)
|                   | — update + motivo_rejeicao
|                   | — writeEvento 'rejeitado'
|                   | — sendMail rejected (com editUrl
|                   |   /afastamentos/editar/{token_edicao})
|
|                                     | — recebe e-mail —————>
|                                     | GET /afastamentos/editar/[token]
|                                     | — isEditAllowed? (só se situacao='rejeitado')
|                                     | — carrega afastamento + lookups + banner motivo
|
|                                     | PATCH /api/public/afastamentos/[token] {...}
|

```

```

— isEditAllowed + canTransition → 'pendente'
— update + situacao=pendente + motivo_rejeicao=null
— writeEvento 'resubmetido'
← inbox reflete o registro resubmetido (volta para pendente)

```

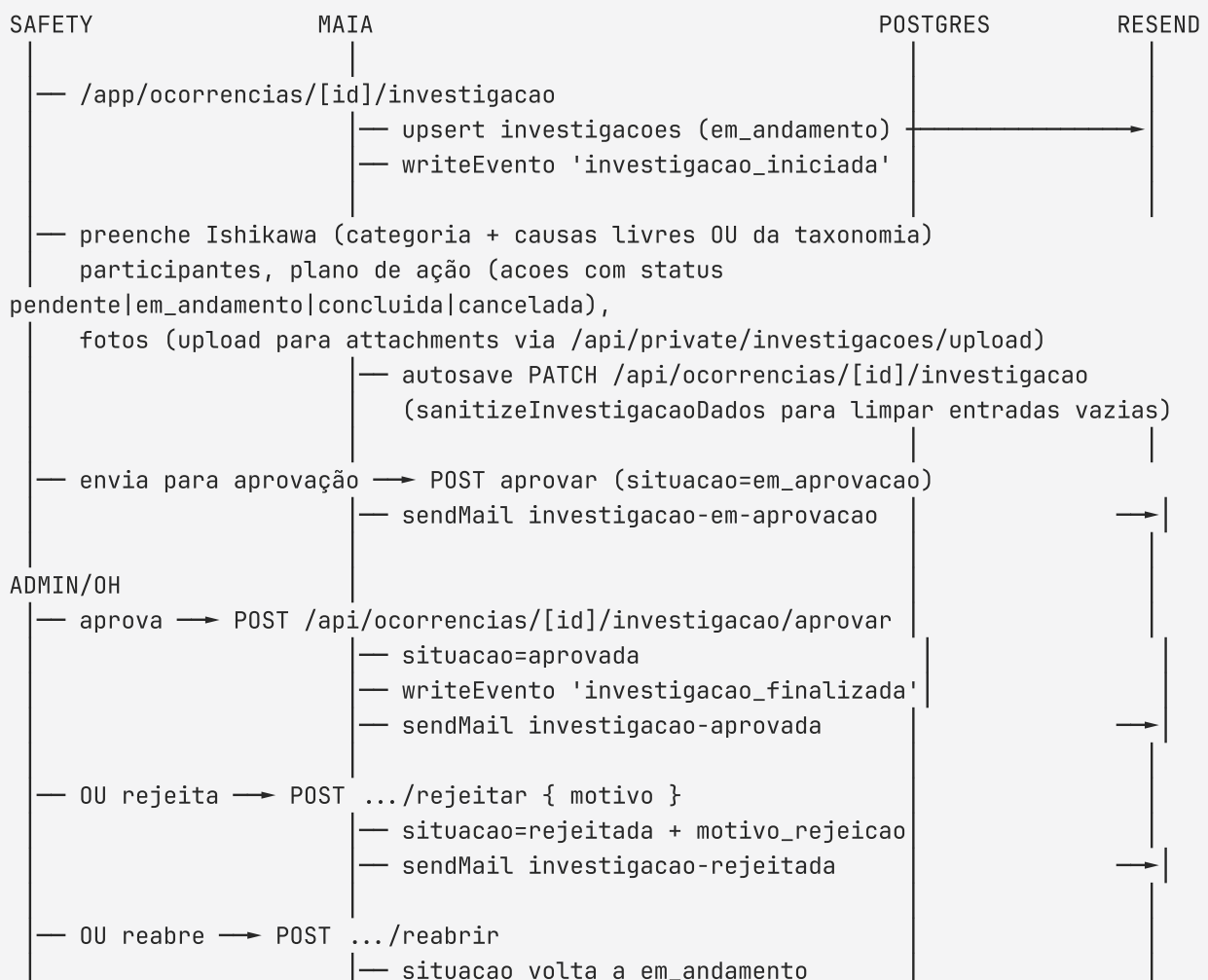
7.4 Submissão de ocorrência + notificação à equipe Safety

Análogo ao 7.1 mas:

- `ocorrencias.situacao` inicia em `aberta`.
- Após o insert, `safety-notify.ts` dispara `sendMail({ template: "ocorrencia-nova-para-safety", ... })` para os e-mails dos membros da equipe `safety`.
- Eventos `ocorrencia_para_safety_enviada` ou `ocorrencia_para_safety_falhou` são gravados conforme resultado.
- O autor recebe `ocorrencia-receipt` com link de status (`/ocorrencias/status/[token]`) e relatório (`/ocorrencias/relatorio/[token]`).

7.5 Investigação Ishikawa (equipe Safety)

State machine (`lib/investigacao-state.ts`): `em_andamento` → `em_aprovacao` → `aprovada` | `rejeitada` | `cancelada`.



A investigação também pode ser preenchida **publicamente por token** em

`/investigacoes/editar/[token]` (modelo Calendly), para envio a peritos externos. O handler `POST /api/public/investigacoes/[token]/submeter` aplica `sanitizeInvestigacaoDados` e `zod-valida` o `jsonb` antes de salvar.

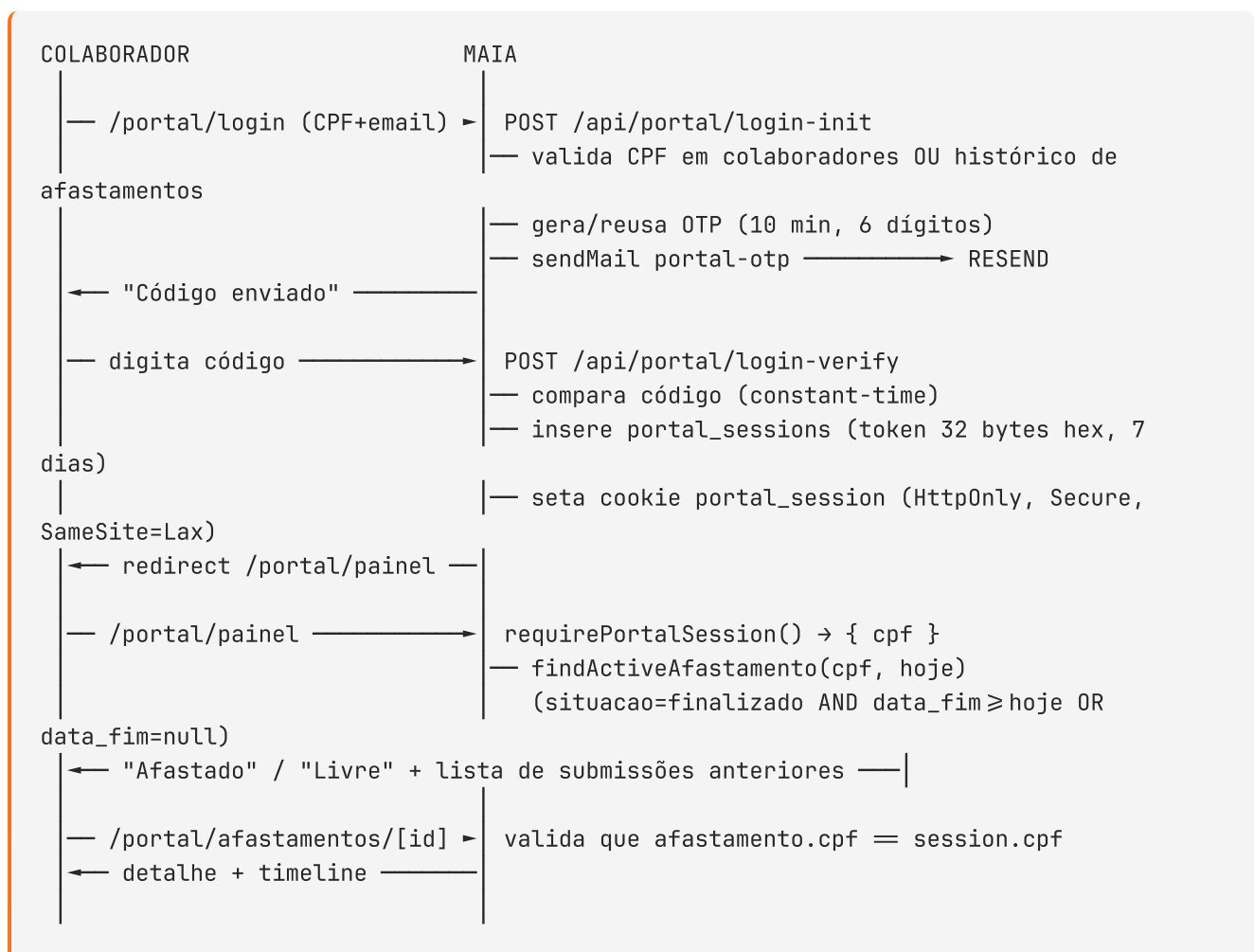
7.6 Comentários em afastamentos

Pequena thread acoplada ao detalhe do afastamento (`/app/afastamentos/[id]`). Acesso restrito a staff autenticado (gate no handler).

- `GET /api/afastamentos/[id]/comentarios` — lista comentários ordenados por `criado_em`.
- `POST /api/afastamentos/[id]/comentarios` — `{ texto, anexos[] }`.
- `PATCH /api/afastamentos/[id]/comentarios/[comentarioId]` — edição (apenas pelo autor; seta `editado_em`).
- `DELETE /api/afastamentos/[id]/comentarios/[comentarioId]` — apenas pelo autor.
- `POST /api/afastamentos/[id]/comentarios/upload` — upload de anexo para `attachments/comentarios/{afastamento_id}/{uuid}`.

Os anexos são guardados como `jsonb` no comentário (`[{ nome, url, content_type, tamanho}]`).

7.7 Portal do colaborador





7.8 Relatório PDF público de ocorrência

GET `/api/public/ocorrencias/relatorio/[token]` (e a página correspondente em `/ocorrencias/relatorio/[token]`).

- Valida `token_publico` em `ocorrencias` e busca a investigação relacionada.
- Renderiza o componente `<InvestigacaoReport>` server-side em HTML.
- `puppeteer-core` + `@sparticuz/chromium` (Chromium serverless) gera o PDF a partir do HTML.
- Retorna `Content-Type: application/pdf` com filename `relatorio-ocorrencia-{serial_id}.pdf`.

A foto do upload de investigação (público) tem endpoint dedicado: GET `/api/public/investigacoes/[token]/foto?path=...` que faz proxy de signed URL do storage.

7.9 Painel de saúde da operação

`/app/painel/saude` (admin-gated) consome GET `/api/saude` → `getSaudeMetrics()` em `lib/dashboard/queries.ts`. Retorna:

- **E-mails falhados (24h):** agrupa eventos `email_enviado` com `dados.error` por `entidade_id`; lista até 5.
- **Fluig falhados (24h):** agrupa eventos `fluig_erro` por `entidade_id`; conta `tentativas`, captura `ultimo_erro`, `ultimo_erro_status`, `ultimo_erro_raw` (JSON da edge function). Cada item abre uma `FluigErrorSheet` com botão **"Retentar agora"** (admin-only).
- **Latência de aprovação (30d):** p50 e p95 (em horas) entre `criado` e `aprovado` por afastamento.
- **Ocorrências por situação:** contagem por `situacao`.
- **Afastamentos por tipo (mês corrente):** top 8 + "Outros".
- **Anexos presentes vs ausentes:** count de `arquivo_url` not null.
- **Threshold de SLA:** `configuracoes_dashboard.config.aprovacao_lenta_horas` (default 24).

7.10 Exportação CSV

GET `/api/relatorios/afastamentos?<filtros>` e `/api/relatorios/ocorrencias?<filtros>` retornam CSV com `Content-Disposition: attachment`. Os filtros aceitos espelham os da lista. Builders em `lib/relatorio/`:

- `csv.ts` — escape e header generation.
- `afastamentos-csv.ts` — colunas: `serial_id`, `situacao`, `tipo`, `colaborador`, `CPF`, `datas`, `empresa`, `unidade`, `decidido_por`, `decidido_em`, `motivo_rejeicao`, `criado_em`.
- `ocorrencias-csv.ts` — colunas equivalentes para ocorrências.

7.11 Notificações in-app

Sino no topo da navegação (`AppNotificationBell`) lê `GET /api/notificacoes` → eventos relevantes ao usuário (mutações em afastamentos/ocorrências/investigações de que ele participa) menos os já marcados em `eventos_lidos`. `POST /api/notificacoes/[id]/read` insere a linha de read-receipt.

8. Integrações externas

8.1 SOC (consulta de cadastro)

- **O que faz:** dado um CPF, retorna `{ nome, setor, cargo, codigo_soc, codigo_empresa_soc, codigo_unidade_soc }`.
- **Onde:** edge function `maia-db/supabase/functions/soc-lookup/index.ts`.
- **API SOC:** URL `https://ws1.soc.com.br/WebSoc/exportadados` recebe parametro (JSON-stringify das credenciais + CPF). Resposta em **ISO-8859-1**, decodificada com `TextDecoder('iso-8859-1')` antes do `JSON.parse`.
- **Quem chama:** `/api/public/afastamentos/lookup-cpf`, acionado pelo componente `<CpfLookup>`.
- **Segredos:** `SOC_EMPRESA_PRINCIPAL`, `SOC_EXPORT_CODE`, `SOC_EXPORT_KEY`.

8.2 Fluig (TOTVS — workflow / folha)

- **O que faz:** para cada afastamento médico aprovado, cria documento (atestado) e inicia o processo `wkfIntegraAtestado`.
- **Onde:** edge function `maia-db/supabase/functions/fluig-push/index.ts`.
- **Protocolo:** SOAP em duas fases:
 1. `ECMDocumentService.createSimpleDocument` — upload base64, devolve `documentId`.
 2. `ECMWorkflowEngineService.startProcess` — inicia o processo, passando `documentIdAtestado` dentro de `<cardData>` (lista `<item><item>key</item><item>value</item></item>`).
- **Mapeamento:** 7 tipos médicos são empurrados (`doenca`, `acidente`, `consulta_medica`, `doacao_sangue`, `realizacao_exames`, `prev_31`, `prev_91`). Demais retornam `{ skipped: true }`.
- **Segurança XML:** campos dinâmicos passam por `xmlEscape()`.
- **Wrapper no app** (`lib/fluig.ts`): converte `FunctionsHttpError` em `{ ok:false, error:{ message, body, status } }` — **nunca lança**, para não derrubar a aprovação.
- **Segredos** (Supabase Secrets): `FLUIG_BASE_URL`, `FLUIG_USERNAME`, `FLUIG_PASSWORD`, `FLUIG_PARENT_DOC_ID`, `FLUIG_PROCESS_ID`.

8.3 Resend (e-mail)

- **SDK:** `resend` v6 (Node, não funciona em edge runtime — envio sempre no `maia-app`).

- **Wrapper:** `lib/mail/send.ts` mantém registro de **14 templates** (ver §10) e dispatcher único `sendMail({ template, to, data })`.
- **Inicialização tardia:** `new Resend(process.env.RESEND_API_KEY!)` é construído dentro de `sendMail()`.
- **Override de dev:** quando `NODE_ENV !== "production"` e o template tem `toUser: true` (recipient é um usuário do sistema com `@seed.local`), o destinatário é trocado por `dev-tests@fapptory.me` e o assunto recebe um prefixo `[DEV → original]`.
- **Segredos:** `RESEND_API_KEY`, `RESEND_FROM_EMAIL`, `RESEND_FROM_NAME`.

8.4 Supabase

- **Auth (staff):** invites via `admin.auth.admin.inviteUserByEmail()` — conteúdo do e-mail vem da configuração de Auth Templates do Supabase Dashboard.
- **Postgres:** três clientes no app: `getSupabaseServer()` (escopado nos cookies do request), `getSupabaseBrowser()` e `getSupabaseAdmin()` (service-role). Tipos gerados em `lib/supabase/database.types.ts` via `supabase gen types typescript --linked` — **nunca editar manualmente**.
- **Storage:** bucket `attachments` (privado). URLs assinadas geradas server-side. Convenções de path:
 - `afastamentos/staging/{uuid}-{nome}` — atestados.
 - `comentarios/{afastamento_id}/{uuid}-{nome}` — anexos de comentário.
 - `investigacoes/{ocorrencia_id}/{uuid}-{nome}` — fotos.
 - `avatars/{user_id}.{ext}` — fotos de perfil.

8.5 PDF (puppeteer + chromium serverless)

- **Onde:** `/api/public/investigacoes/[token]/pdf` e potencialmente `/ocorrencias/relatorio/[token]`.
- **Como:** renderiza o componente `<InvestigacaoReport>` em HTML, passa para `puppeteer-core` que usa `@sparticuz/chromium` (binário Chromium otimizado para serverless Vercel/AWS Lambda).
- **Saída:** PDF A4 com cabeçalho e rodapé customizados.

9. Rotas e endpoints

Páginas (37)

Públicas (sem login)

Caminho	Conteúdo
/	Linktree público (logo, atalhos)
/forms/afastamentos	Formulário público de afastamento

Caminho	Conteúdo
/forms/ocorrencias	Formulário público de ocorrência
/afastamentos/editar/[token]	Reenvio de afastamento rejeitado
/afastamentos/status/[token]	Consulta de status por protocolo
/ocorrencias/status/[token]	Consulta de status de ocorrência
/ocorrencias/relatorio/[token]	Visualização do relatório de ocorrência (HTML + link PDF)
/investigacoes/editar/[token]	Preenchimento de investigação Ishikawa por perito externo

Auth (staff)

Caminho	Conteúdo
/login	Login com email/senha
/forgot-password	Requisita reset
/update-password	Seta nova senha (após invite ou recovery)

Portal do colaborador

Caminho	Gate	Conteúdo
/portal/login	público	OTP (CPF + e-mail)
/portal/cadastro	público	Cadastro inicial de colaborador
/portal/painel	portal_session cookie	Status + histórico de afastamentos
/portal/afastamentos/[id]	portal_session cookie	Detalhe de submissão (do próprio CPF)

Área interna (staff)

Caminho	Gate	Conteúdo
/app/painel	autenticado	Contadores + atividade recente
/app/painel/saude	admin	Painel de saúde da operação (métricas + retry Fluig)
/app/perfil	autenticado	Nome, troca de senha, avatar

Caminho	Gate	Conteúdo
/app/afastamentos	autenticado	Lista com filter rail + exportação CSV
/app/afastamentos/ativos	autenticado	Filtro pronto: finalizados em curso
/app/afastamentos/[id]	autenticado	Detalhe + timeline + comentários
/app/afastamentos/aprovacoes	equipe oh	Inbox redimensionável (lista + detalhe + ações)
/app/ocorrencias	autenticado	Lista
/app/ocorrencias/[id]	autenticado	Detalhe
/app/ocorrencias/[id]/investigacao	equipe safety	Formulário Ishikawa completo
/app/investigacoes	equipe safety	Lista de investigações
/app/admin	admin	Hub
/app/admin/usuarios	admin	Listar + convidar + toggle + excluir
/app/admin/equipes	admin	Gestão de membros
/app/admin/configuracoes	admin	E-mail da folha
/app/admin/empresas	admin	CRUD
/app/admin/unidades	admin	CRUD
/app/admin/afastamento-tipos	admin	CRUD
/app/admin/colaboradores	admin	Consulta de cadastro por CPF
/app/admin/investigacao/categorias	admin	CRUD das 6Ms
/app/admin/investigacao/causas	admin	CRUD das causas seedadas
/app/admin/investigacao/graus	admin	CRUD da escala de gravidade

Route Handlers (61)

Públicos

Método	Endpoint	Resumo
POST	/api/public/afastamentos	Submete afastamento, dispara recibo + folha-auto-accept
POST	/api/public/afastamentos/upload	Upload do anexo (pdf/jpg/png/webp, ≤10 MB)
GET	/api/public/afastamentos/upload/preview	Signed URL de preview do anexo recém-enviado
POST	/api/public/afastamentos/lookup-cpf	Invoca soc-lookup + cruza com empresas/unidades
GET	/api/public/afastamentos/ocorrencias-disponiveis	Lista ocorrências do CPF para bind opcional
GET	/api/public/afastamentos/[token]	Carrega afastamento por token de edição
PATCH	/api/public/afastamentos/[token]	Resubmissão (gate isEditAllowed)
POST	/api/public/ocorrencias	Registra ocorrência + recibo + notifica safety
GET	/api/public/investigacoes/[token]	Carrega investigação por token público
POST	/api/public/investigacoes/[token]/submeter	Salva/finaliza investigação
GET	/api/public/investigacoes/[token]/foto	Signed URL para foto da investigação
GET	/api/public/investigacoes/[token]/pdf	Gera PDF do relatório (puppeteer)

Portal do colaborador

Método	Endpoint	Resumo
POST	/api/portal/login-init	Valida CPF, gera OTP (10 min), envia e-mail
POST	/api/portal/login-verify	Valida OTP, cria session (7 dias), seta cookie
POST	/api/portal/logout	Deleta session + limpa cookie

Autenticados (staff)

Método	Endpoint	Resumo
GET	/api/me	Usuário + equipes + flags
POST	/api/me/avatar	Upload de avatar (JPEG/PNG, redimensionado)
GET	/api/notificacoes	Eventos relevantes não lidos pelo usuário
POST	/api/notificacoes/[id]/read	Marca como lido (insert em eventos_lidos)
GET	/api/afastamentos	Lista com filtros (situacao, tipo, empresa_id, unidade_id, cpf, from, to, q)
GET	/api/afastamentos/[id]	Detalhe
POST	/api/afastamentos/[id]/aprovar	Admin OU oh; push Fluig + update + e-mails
POST	/api/afastamentos/[id]/rejeitar	Admin OU oh; motivo obrigatório; e-mail com editUrl
POST	/api/afastamentos/[id]/cancelar	Admin only; pendente/rejeitado → cancelado
PATCH	/api/afastamentos/[id]/editar	Admin OU oh; edição administrativa pontual
POST	/api/afastamentos/[id]/fluig/retry	Admin only; retenta push Fluig após erro
GET	/api/afastamentos/[id]/comentarios	Lista comentários
POST	/api/afastamentos/[id]/comentarios	Cria comentário (texto + anexos)
PATCH	/api/afastamentos/[id]/comentarios/[comentarioId]	Edita (autor only)
DELETE	/api/afastamentos/[id]/comentarios/[comentarioId]	Apaga (autor only)
POST	/api/afastamentos/[id]/comentarios/upload	Upload de anexo para o comentário
GET	/api/ocorrencias	Lista

Método	Endpoint	Resumo
GET	/api/ocorrencias/[id]	Detalhe + investigação
POST	/api/ocorrencias/[id]/investigacao	Upsert da investigação (autosave + submit)
POST	/api/ocorrencias/[id]/investigacao/aprovar	Admin OU oh; aprova investigação
POST	/api/ocorrencias/[id]/investigacao/rejeitar	Admin OU oh; rejeita com motivo
POST	/api/ocorrencias/[id]/investigacao/reabrir	Reabre investigação aprovada/rejeitada
GET	/api/eventos/[entityType]/[entityId]	Histórico unificado
GET	/api/relatorios/afastamentos	Exporta CSV com filtros
GET	/api/relatorios/ocorrencias	Exporta CSV com filtros
GET	/api/saude	Métricas operacionais (admin only)
POST	/api/auth/forgot-password	Wrapper para auth.resetPasswordForEmail
POST	/api/auth/magic-link	Alternativa de acesso para staff (link único)
GET	/api/private/anexos/preview	Signed URL de anexo (autenticado)
POST	/api/private/investigacoes/upload	Upload de fotos para investigação

Admin

Método	Endpoint	Resumo
GET/POST	/api/admin/usuarios	Listar / convidar (Supabase invite + cleanup órfão)
PATCH/DELETE	/api/admin/usuarios/[id]	Renomear, toggle admin/ativo / excluir (auth + public)

Método	Endpoint	Resumo
GET	/api/admin/equipes	Equipes + membros
POST/DELETE	/api/admin/equipes/[id]/membros	Adicionar / remover membro
GET/PATCH	/api/admin/configuracoes	Singleton
GET/POST	/api/admin/empresas + PATCH /[id]	CRUD
GET/POST	/api/admin/unidades + PATCH /[id]	CRUD
GET/POST	/api/admin/afastamento-tipos + PATCH /[id]	CRUD
GET	/api/admin/colaboradores	Lista paginada
GET	/api/admin/colaboradores/[cpf]	Detalhe + histórico de afastamentos
GET/POST	/api/admin/investigacao/categorias + [id]	CRUD
GET/POST	/api/admin/investigacao/causas + [id]	CRUD
GET/POST	/api/admin/investigacao/geraus + [id]	CRUD

Edge functions (2)

Função	Método	Body	Responsabilidade
soc-lookup	POST	{ cpf }	Consulta SOC, devolve perfil do colaborador
fluig-push	POST	FluigPushPayload	Cria documento + inicia processo no Fluig

10. Sistema de e-mails

14 templates registrados em lib/mail/send.ts + 2 templates aplicados via Supabase Dashboard.

Template	Disparado por	Quem recebe	toUser
afastamento-receipt	POST /api/public/afastamentos	autor (email_remetente)	false
folha-auto-accept	POST /api/public/afastamentos (finalizado direto)	configuracoes.email_folha	true
afastamento-approved	POST /api/afastamentos/[id]/aprovar	autor	false
folha-approved-medical	POST /api/afastamentos/[id]/aprovar	configuracoes.email_folha	true
afastamento-rejected	POST /api/afastamentos/[id]/rejeitar (com editUrl)	autor	false
ocorrencia-receipt	POST /api/public/ocorrencias	autor	false
ocorrencia-nova-para-safety	POST /api/public/ocorrencias (notifica equipe)	membros da equipe safety	true
investigacao-em-aprovacao	POST /api/ocorrencias/[id]/investigacao (submit)	admin + equipe oh	true
investigacao-aprovada	POST /api/ocorrencias/[id]/investigacao/aprovar	equipe safety + autor da investigação	true
investigacao-rejeitada	POST /api/ocorrencias/[id]/investigacao/rejeitar	equipe safety + autor	true
portal-otp	POST /api/portal/login-	colaborador (e-mail informado)	false

Template	Disparado por	Quem recebe	toUser
	init		
magic-link	POST /api/auth/magic-link	staff (alternativa de login)	true
user-invite *	Supabase Dashboard → Auth → Invite user	Supabase dispara automaticamente	true
password-reset *	Supabase Dashboard → Auth → Reset password	Supabase dispara automaticamente	true

Estrutura: cada template é uma função TypeScript `(data) ⇒ string` que monta HTML puro com CSS inline. Componentes compartilhados:

- `emails/_layout.ts` (`layout(title, bodyHtml)`)
- `emails/_record-table.ts` (`recordTable(rows)`)
- `emails/_escape.ts` (`escapeHtml`)
- `emails/tokens.ts` (paleta de cores)

Override de desenvolvimento: quando `NODE_ENV !== "production"` e `toUser: true`, o destinatário é trocado pelo `DEV_RECIPIENT` (`dev-tests@fapptory.me`) e o assunto recebe prefixo `[DEV → original-email]`. Templates com `toUser: false` (recipiente externo via dados do formulário) são enviados como-is.

Por que HTML puro? Para manter dependências mínimas (sem React Email + render) e garantir compatibilidade com qualquer cliente de e-mail sem depender de runtime de serialização.

* `user-invite` e `password-reset` **não** são despachados via `lib/mail/send.ts`. Eles geram HTML estático via `npm run scripts` em `scripts/output/` que é colado em Auth → Email Templates no Supabase Dashboard.

`emails/relatorio-pronto.ts` existe no diretório mas não está registrado no dispatcher — está reservado para uso futuro quando relatórios pesados precisarem ser despachados de forma assíncrona.

11. Auditoria (eventos)

A tabela `eventos` é a única fonte de auditoria. Cada Route Handler que muda estado escreve um evento via `lib/eventos.ts → writeEvento(...)`. A tabela acessória `eventos_lidos` rastreia read-receipts por usuário (notificações in-app).

Tipos atuais (`EventoType` — 13 valores)

evento	Quando
criado	Submissão pública de afastamento/ocorrência
aprovado	Aprovação OH (afastamento) ou aprovação de investigação
rejeitado	Rejeição OH ou rejeição de investigação
resubmetido	Autor reenvia via link de edição
cancelado	Admin cancela afastamento (ou investigação)
editado	Edição administrativa pontual (PATCH <code>/api/afastamentos/[id]/editar</code>)
fluig_enviado	Push Fluig retorna <code>ok: true</code>
fluig_erro	Push Fluig falha; payload contém <code>error.message</code> , <code>error.body</code> , <code>error.status</code> , <code>retry: bool</code>
email_enviado	Qualquer template (sucesso ou erro — <code>dados.error</code> diferencia)
investigacao_iniciada	Primeira escrita em <code>investigacoes</code> (upsert quando entra <code>em_andamento</code>)
investigacao_finalizada	Investigação aprovada (<code>situacao=aprovada</code>)
ocorrencia_para_safety_enviada	E-mail à equipe safety despachado com sucesso
ocorrencia_para_safety_falhou	Falha no envio à equipe safety

Todos os eventos carregam:

- `tipo_entidade` (`afastamento` | `ocorrencia` | `investigacao`)
- `entidade_id`
- `autor_id` (quando autenticado; null para fluxos públicos)
- `dados_jsonb` (motivo, response, error, retry, etc.)

A timeline na UI (`components/detail/timeline-events.tsx`) faz fetch de `/api/eventos/[entityType]/[entityId]` e renderiza com rótulos PT-BR via `lib/eventos-format.ts`.

Read-receipts

`eventos_lidos` (`usuario_id`, `evento_id` — PK composta) registra leitura individual. O sino de notificações lê apenas eventos relevantes ao usuário (e ainda não lidos), populando o badge no top nav. `POST /api/notificacoes/[id]/read` insere a linha — ON DELETE CASCADE em ambos os lados garante que o usuário pode marcar tudo como lido sem deixar lixo.

12. Design system

Tokens

Definidos em `app/tokens.css` e expostos ao Tailwind 4 via `@theme inline` em `app/globals.css`:

- **Cores:** `bg`, `bg-subtle`, `fg`, `fg-muted`, `border`, `primary` (azul ENGEKO), `primary-fg`, `primary-hover`, `success`, `warning`, `danger`, `info`, `danger-soft`.
- **Espaçamento:** `--space-1` a `--space-12`.
- **Tipografia:** `--font-sans` (Inter, system-ui), `--font-mono` (JetBrains Mono). Escala `--text-xs` a `--text-3xl`.
- **Radius:** `sm` / **md (cap)** / `lg`. **Sem `rounded-full` em retângulos** — apenas em avatares circulares.
- **Sombra:** `sm` / `md` / `lg`.

Componentes

- **shadcn/ui** para primitivas (`button`, `label`, `separator`, `textarea`, `resizable`). Instalados sob demanda com `npx shadcn@latest add <name>`.
- **@base-ui/react** para componentes que o shadcn não cobre bem ou onde precisamos da API funcional do `SelectValue` (`(value) ⇒ ReactNode`). Adotado para o form Ishikawa, sheets e dropdowns mais sofisticados.
- **Custom:** `components/forms/*`, `components/afastamentos/*`, `components/investigacoes/*`, `components/saude/*`, `components/portal/*`, `components/detail/*`, `components/layout/*`, `components/admin/crud-table.tsx`.
- **Convenção:** classes Tailwind diretas em JSX; cores via `bg-[var(--color-success)]`, `text-[var(--color-danger)]` quando o token não está exposto no `@theme`.

Convenções específicas

- **Cap em rounded-md:** nenhum retângulo deve usar `rounded-xl` (existentes ficam grandfathered) nem `rounded-full`. Apenas avatares circulares usam `rounded-full`.
- **PT-BR em tudo:** rótulos, mensagens de erro, breadcrumbs, placeholders.

E-mail

Tokens próprios em `emails/tokens.ts` — espelham as cores do app mas em hex literal para inlining seguro (sem depender de variáveis CSS no e-mail).

13. Testes

Vitest — 35 arquivos unitários

Cobertura por área:

Arquivo	Cobertura
permissions.test.ts	isAdmin , isInEquipe
validation.test.ts	AfastamentoInputSchema (CPF, e-mail, emissor, etc.)
ocorrencia-validation.test.ts	OcorrenciaInputSchema
auth-schemas.test.ts	Schemas zod de OTP, login, reset
auth-errors.test.ts	Mensagens normalizadas
eventos.test.ts	writeEvento com mocks
eventos-format.test.ts	Rótulos PT-BR por tipo
afastamento-state.test.ts	canTransition (6 transições válidas + denials)
afastamento-date.test.ts	Cálculo de data_fim por tipo
edit-token.test.ts	isEditAllowed
ocorrencia-state.test.ts	States + labels de tipo
investigacao-dados-schema.test.ts	Schema jsonb + sanitizeInvestigacaoDados
investigacao-jsonb-fk-check.test.ts	Validação de FKs internas (categoria_id, grau_id, causa_id)
investigacao-permissions.test.ts	Quem pode aprovar/rejeitar/reabrir
investigacao-step-gates.test.ts	Gates por step do form
portal-auth.test.ts	requirePortalSession (cookie válido/inválido/expirado)
portal-session.test.ts	CRUD de sessions
portal-login-init.test.ts	Validação CPF + e-mail + OTP reuse
portal-status.test.ts	findActiveAfastamento (8 cenários)
safety-notify.test.ts	Notificação à equipe safety
dashboard-queries.test.ts	Funções de métricas
filter-rail.test.ts	Builder de query
colaborador-summary.test.ts	Display de nome/CPF
status-pill.test.ts	Variantes por situacao
painel-hero-content.test.ts	Hero card por papel

Arquivo	Cobertura
<code>nav.test.ts</code> / <code>public-nav.test.ts</code>	Navegação
<code>public-links.test.ts</code>	URL builders
<code>greeting.test.ts</code>	Saudação contextual
<code>fmt-date.test.ts</code>	Formatadores
<code>fapptory.test.ts</code>	Branding utility
<code>afastamentos-csv.test.ts</code> / <code>ocorrencias-csv.test.ts</code> / <code>relatorio-csv.test.ts</code>	Exportação CSV
<code>migrate-afastamentos.test.ts</code>	Script de migração de legado

Comando: `npx vitest run`.

Playwright — 5 cenários E2E

Arquivo	Cobertura
<code>happy-path.spec.ts</code>	Submissão pública → autofill SOC → aprovação OH → toast
<code>auth-pages.spec.ts</code>	Páginas de login/forgot/update-password
<code>painel.spec.ts</code>	Painel interno (kpis, atividade recente)
<code>phase-7-saude.spec.ts</code>	Painel de saúde + retry Fluig
<code>public-landing.spec.ts</code>	Landing pública e navegação

Pré-requisitos: ambiente deployado (Supabase cloud, edge functions ativas, Vercel, e-mail real) + env vars `E2E_BASE_URL`, `E2E_OH_EMAIL`, `E2E_OH_PASSWORD`, `E2E_TEST_CPF`.

Comando: `npx playwright test`.

Deno — 4 testes (edge function mapping)

`maia-db/supabase/functions/_shared/fluig-mapping.test.ts` — `mapTipoToFluigCode` para tipos médicos, não-médicos e desconhecidos.

Comando: `cd maia-db && deno test supabase/functions/_shared/fluig-mapping.test.ts`.

Smoke psql (RLS)

`maia-db/supabase/tests/rls.sql` valida que admin e membro `oh` veem o afastamento de teste, mas usuário aleatório vê zero. Roda contra Postgres local (após `make db-reset`).

Comando: `psql "$(supabase status -o env | grep DB_URL | cut -d= -f2-)" -f supabase/tests/rls.sql`.

14. Variáveis de ambiente

maia-app (Vercel e/ou `.env.local`)

Variável	Lugar	Uso
<code>NEXT_PUBLIC_SUPABASE_URL</code>	Vercel + local	URL base do projeto Supabase
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Vercel + local	Chave anon (client/server)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Vercel + local	Service-role, nunca exposta ao browser
<code>RESEND_API_KEY</code>	Vercel + local	Resend
<code>RESEND_FROM_EMAIL</code>	Vercel + local	Ex: <code>nr-maia@heizen.io</code>
<code>RESEND_FROM_NAME</code>	Vercel + local	Ex: <code>No-Reply MAIA</code>
<code>NEXT_PUBLIC_APP_BASE_URL</code>	Vercel + local	Ex: <code>https://maia.engeko.com.br</code> . Usado em links de e-mail.
<code>NEXT_PUBLIC_APP_VERSION</code>	injetada	Set automaticamente pelos scripts <code>dev/build/start</code>

maia-db (Supabase secrets via `supabase secrets set`)

Variável	Usada por	Conteúdo
<code>SOC_EMPRESA_PRINCIPAL</code>	<code>soc-lookup</code>	Código da empresa-resposta no SOC
<code>SOC_EXPORT_CODE</code>	<code>soc-lookup</code>	Código da exportação
<code>SOC_EXPORT_KEY</code>	<code>soc-lookup</code>	Chave de autenticação SOC
<code>FLUIG_BASE_URL</code>	<code>fluig-push</code>	URL base do webdesk Fluig
<code>FLUIG_USERNAME</code>	<code>fluig-push</code>	Usuário do serviço
<code>FLUIG_PASSWORD</code>	<code>fluig-push</code>	Senha do serviço
<code>FLUIG_PARENT_DOC_ID</code>	<code>fluig-push</code>	ID do documento pai para uploads

Variável	Usada por	Conteúdo
FLUIG_PROCESS_ID	fluig-push	ID do processo (wkfIntegraAtestado)
FLUIG_COMPANY_ID	fluig-push	ID numérico da empresa no Fluig (startProcess)

15. Deploy

Sequência canônica

1. Criar projeto Supabase (cloud) — copiar Project Ref .
2. Vincular maia-db e publicar schema (todas as 20 migrations):

```
cd /Users/heizen/DEV/maia-db
supabase login
supabase link --project-ref <ref>
make db-push      # ou: supabase db reset --linked (destrutivo)
```

3. Regenerar tipos no maia-app após qualquer migração:

```
cd /Users/heizen/DEV/maia-app
supabase gen types typescript --linked > lib/supabase/database.types.ts
```

Esse arquivo é **auto-gerado** — nunca editar manualmente.

4. Deployar edge functions:

```
cd /Users/heizen/DEV/maia-db
make functions-deploy
```

5. Configurar segredos do maia-db (supabase secrets set ...) — todos os SOC_* e FLUIG_*.
6. Criar usuário admin inicial:
 - Supabase Dashboard → Auth → Add user (email + senha).
 - SQL Editor: insert into usuarios (id, nome, email, administrador, ativo) values ('<auth-uid>', 'Admin', 'admin@engeko.com.br', true, true);
7. Substituir placeholders de seed (se aplicável):
 - Editar seed.sql com dados reais ou rodar manualmente via SQL Editor.
 - Substituir lib/data/cids.json pela lista CID-10 fornecida pela SO.
8. Configurar SMTP customizado (Project Settings → Auth → SMTP) com credenciais Resend.

9. **Configurar Auth Templates** (Authentication → Email Templates) colando o HTML de `scripts/output/user-invite.html` e `password-reset.html`.

10. **Criar projeto Vercel:**

```
cd /Users/heizen/DEV/maia-app
npx vercel link
npx vercel --prod
```

11. **Domínio customizado** no Vercel (ex: `maia.engeko.com.br`) — esperar SSL.

12. **Smoke manual end-to-end:**

- `/forms/afastamentos` → submeter (medical).
- Login como admin → convidar usuário OH real → adicionar à equipe `oh`.
- OH aprova, confirma e-mails e push Fluig.
- Submeter ocorrência → preencher investigação Ishikawa → aprovar.
- Colaborador autentica no portal via OTP.

13. **Smoke automatizado** (opcional):

```
E2E_BASE_URL=https://maia.engeko.eng.br \
E2E_OH_EMAIL=... E2E_OH_PASSWORD=... \
E2E_TEST_CPF=... \
npx playwright test
```

Quando algo dá errado

- **Edge function 502:** cheque `supabase secrets list` — algum `SOC_*` ou `FLUIG_*` faltando. Detalhes do erro aparecem no painel de saúde via `FluigErrorSheet` (corpo da resposta da edge function).
- **Resend rejeita o e-mail:** confirme que o domínio remetente está verificado no painel do Resend.
- **Página em loop de redirect para /login:** `NEXT_PUBLIC_SUPABASE_URL` ou `NEXT_PUBLIC_SUPABASE_ANON_KEY` ausente — `proxy.ts` falha em criar o client.
- **"Could not find the 'X' column":** migração nova não aplicada ou tipos do Supabase desatualizados — rodar `supabase db push` + regenerar `database.types.ts`.
- **make db-reset quebra localmente:** Docker do Supabase precisa estar rodando (`supabase start`).
- **Convite admin não cria linha em usuarios:** a função usa `inviteUserByEmail` + `insert` em duas etapas; se o `insert` falhar, removemos o `auth.user` órfão. Confira o motivo do erro retornado em `error.message`.
- **Excluir usuário falha:** o handler DELETE remove `public.usuarios` antes de `auth.users`. Se o `auth.users` delete falhar, retorna `{ ok: true, warning: "..."` } — o usuário sumiu da app mas precisa ser limpo manualmente para reinvite.

16. Convenções

- **PT-BR para tudo que é texto humano:** identificadores de código mantêm convenções técnicas (camelCase), mas linhas de texto, comentários explicativos, README, UI, mensagens de erro e e-mails são em português.
- **Mínimo de dependências:** shadcn + @base-ui/react + Tailwind cobrem a UI. Sem Radix individual, sem React Email, sem outras libs de componentes.
- **npm:** nunca pnpm/yarn/bun. Lockfile commitado: package-lock.json.
- **Migrações vivem no maia-db:** nunca criar SQL no maia-app, nunca rodar SQL ad-hoc no Dashboard Supabase. Para mudar schema, criar @NN_<nome>.sql em maia-db/supabase/migrations/.
- **Migrações são imutáveis depois de deployadas:** para mudar schema, criar uma migration nova (não editar antiga).
- **database.types.ts é auto-gerado:** rodar supabase gen types typescript --linked após migrações; nunca editar manualmente.
- **Service-role só no servidor:** nunca importar lib/supabase/admin.ts em código com "use client".
- **Permission check antes do admin client:** identificar via getSupabaseServer() → checar permissão na tabela usuarios / equipe_usuarios → só então usar getSupabaseAdmin() para mutar.
- **Sempre logar evento em mutações:** cada update de afastamento/ocorrência/investigação tem um writeEvento(...) correspondente.
- **Erros de e-mail não derrubam o fluxo:** envoltos em try/catch, registrados como eventos.email_enviado com dados.error.
- **Erros de Fluig não derrubam a aprovação:** pushToFluig nunca lança; aprovação prossegue, erro fica em eventos.fluig_erro para retry manual.
- **Erros de update derrubam o fluxo:** capturar upErr e retornar 500. Eventos só após confirmação do update.
- **Cap em rounded-md:** nenhum retângulo deve usar rounded-full. Avatares circulares são a única exceção.
- **Portal isolado de Supabase Auth:** nunca relacionar colaboradores.cpf com auth.users.id. Os dois mundos são separados por design.

17. Backlog e oportunidades de melhoria

Itens entregues em v1.0.0 (anteriormente no backlog)

Item	Status
Formulário Ishikawa de investigação	Entregue — taxonomia 6Ms + 83 causas curadas + plano de ação + fotos

Item	Status
Preview de anexos (signed URL endpoint)	Entregue — <code>/api/private/anexos/preview</code> + variantes públicas por token
Painel de saúde da operação	Entregue — <code>/app/painel/saude</code> + <code>/api/saude</code>
Página "minhas submissões" para o colaborador	Entregue — portal OTP em <code>/portal/painel</code>
Exportação CSV de afastamentos/ocorrências	Entregue — <code>/api/relatorios/*</code>
Retry manual do Fluig após falha	Entregue — <code>/api/afastamentos/[id]/fluig/retry</code> + <code>FluigErrorSheet</code>
Relatório PDF público de ocorrência	Entregue — puppeteer-core + chromium serverless
Notificações in-app	Entregue — sino + <code>eventos_lidos</code>
Comentários em afastamentos	Entregue — thread com anexos
Avatar e perfil do usuário	Entregue — <code>/app/perfil</code> + <code>/api/me/avatar</code>

Mantidos como deferidos

Item	Por que ficou de fora
CID-10 completo (~14k linhas)	Aguardando lista oficial da SO
Lista real de unidades / contratos	Aguardando seed da ENGEKO
OAuth providers (Google)	Out of scope para v1
Retry automático do Fluig (worker/cron)	v1 oferece retry manual no painel; automação fica para v1.x
Antivírus em anexos	LGPD/jurídica — discutir com cliente
Rate limiting nos endpoints públicos	Cloudflare/Vercel WAF na frente do domínio quando necessário
PWA / instalação mobile	Funcional no browser; PWA não foi pedido
Multi-tenant	Single-tenant é a decisão de v1; pivot possível com <code>tenant_id</code>

Oportunidades que valem discussão

17.1 Pequenas vitórias

- **Singleton do client Resend:** hoje `new Resend(...)` é construído a cada `sendMail()` — ok para volume baixo, mas vale promover para módulo-level lazy.
- **Validação de UUID nos handlers:** a maioria dos `[id]/route.ts` confia que o Supabase rejeita IDs malformados. Adicionar `z.string().uuid().parse(id)` (ou o `UuidRegex` permissivo já estabelecido) deixa o 400 explícito.
- **Limites de upload:** hoje validamos `size ≤ 10MB` e MIME em uma única route handler. Vale adicionar validação de magic bytes (ou antivírus via worker).
- **Token de edição não expira:** estável para sempre — vale documentar para auditoria que `token_edicao` só funciona enquanto `situacao = 'rejeitado'`.

17.2 Médio prazo

- **Soft-locks contra race:** aprovação concorrente em janelas múltiplas pode causar dois updates. Escrever o update com `where situacao = 'pendente'` (optimistic lock) blinda o cenário.
- **Logs estruturados:** edge functions e route handlers só registram em `console.log/error`. Vale plugar um sink (Logflare, Better Stack) e padronizar JSON com `request_id`, `user_id`, `route`, `duration_ms`.
- **Histórico de motivos de rejeição:** `motivo_rejeicao` é sobrescrito no PATCH/resubmit. A timeline de eventos guarda histórico, mas vale uma tabela ou aceitar que o evento é o source-of-truth.
- **Reabertura de afastamento finalizado:** state machine não permite. Se a ENGEKO pedir, considerar `finalizado → pendente` com permissão admin-only e motivo obrigatório.
- **Retry automático do Fluig:** hoje o admin precisa retentar manualmente. Vale um cron que repare afastamentos com `eventos.fluig_erro` mais recente que `eventos.fluig_enviado`.

17.3 Maior fôlego

- **Mobile-first / PWA:** o formulário público é onde o colaborador mais entra. Instalação como PWA + reset por SMS (Twilio) seriam valor real.
- **Multi-empresa (controlado):** se a ENGEKO virar holding e separar empresas-filhas operacionalmente, adicionar `tenant_id` em `usuarios / equipes / afastamentos / ocorrencias` e ajustar políticas RLS. Estimativa: ~2 dias com este código como base.
- **Capacidades granulares:** o modelo atual é binário (admin/equipe). Conforme novos times surjam, considerar uma tabela `permissoes` (`nome` + `equipe_id`).

Pontos de risco operacional

1. **CNPJ + unidades + CIDs ainda são placeholders.** Antes do go-live final, substituir. Sugestão: script `scripts/seed-engeko.ts` lendo YAML/CSV controlado pela SO.
2. **Sem rate limiting nos endpoints públicos.** `/forms/*` e `/api/public/*` aceitam qualquer volume. Cloudflare/Vercel WAF resolveriam.
3. **Token de edição não expira.** Estável por design (Calendly-style); funciona apenas enquanto `situacao = 'rejeitado'`.
4. **Anexos não são validados por antivírus.** Para ambiente médico pode virar exigência LGPD.
5. **Push Fluig sem retry automático.** v1 oferece retry manual; automação está no backlog.

Esta documentação reflete o estado do sistema na entrega **v1.0.0** em **2026-05-19**. Atualizar quando schema, fluxos ou integrações mudarem.

18. Migração de anexos legados (2026-05)

Contexto

O sistema anterior armazenava anexos de afastamentos em múltiplas origens: bucket Supabase do projeto legado (`zgdemniuryzfohgxdafu`), links públicos de ClickUp/Fillout, e arquivos que foram baixados manualmente em zips quando o storage excedia a cota permitida. Com a migração para o maia-app, os registros foram importados mantendo o campo `arquivo_url` com a URL completa original (`https:// ...`) em vez de um caminho relativo ao bucket atual.

Problema

O proxy de preview privado (`/api/private/anexos/preview`) e o proxy público de investigações (`/api/public/investigacoes/preview`) esperam caminhos relativos ao bucket `attachments` (ex.: `afastamentos/staging/ ...`). URLs completas externas causavam erro `bad_path` ao tentar visualizar qualquer anexo nesses ~16k registros.

Solução aplicada

Três fases de migração executadas em ordem, todas idempotentes (re-executáveis sem risco de duplicação):

- 1. Fase `external`** — download direto das URLs ClickUp/Fillout ainda acessíveis via HTTP, upload para `attachments/afastamentos/legacy/<filename>`, atualização do `arquivo_url` no banco. Resultado: ~4.239 arquivos recuperados; ~1.773 com 403 (links expirados, irrecuperáveis).
- 2. Fase `pool`** — upload a partir de pasta local (`scripts/attachment-pool/`) populada com os zips de backup e com download do bucket legado via `scripts/download-legacy-bucket.ts`. Resultado: ~6.161 arquivos recuperados.
- 3. Fase `supabase`** — tentativa de fetch das URLs legadas ainda vivas no bucket público do projeto antigo. Descontinuada em favor de popular o pool com os arquivos do bucket legado e re-rodar a fase `pool`.

Total recuperado: ~10.400 anexos. Os ~3.024 restantes (links expirados sem backup disponível) foram aceitos como perda irreversível — situação comum em migrações com múltiplas mudanças de plataforma ao longo do tempo.

Estrutura atual

Todos os novos anexos de afastamentos são salvos como caminhos relativos (`afastamentos/<env>/<slug>`) no bucket `attachments` do projeto atual, servidos exclusivamente via proxy autenticado. O script `scripts/migrate-attachments.ts` permanece no repositório para uso futuro caso novos lotes de URLs legadas sejam identificados.



HEIZEN TECNOLOGIA LTDA.
CNPJ 47.624.793/0001-83
fapptory.me · hq@fapptory.me